

Optimization

From fruit and state to millions of titles

Gregory S. DeLozier, PhD

The Small Table

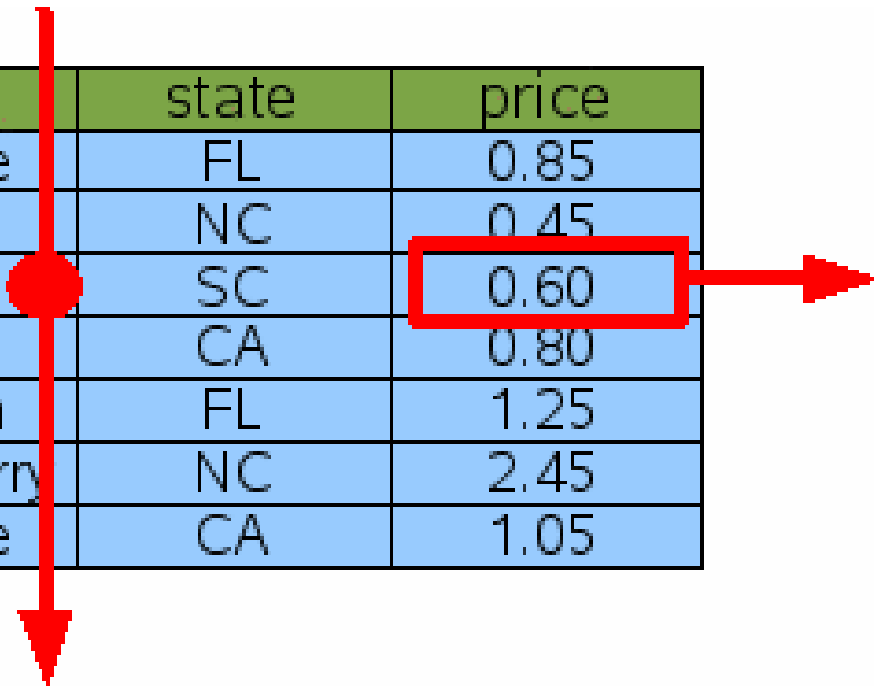
```
create table fruitsforsale (  
    fruit text, state text, price real  
);
```

Seven rows. Oranges from Florida and California.

Follow the rows through the SQLite illustrations.

A Full Table Scan

```
select price from fruitsforsale  
where fruit = 'Peach';
```



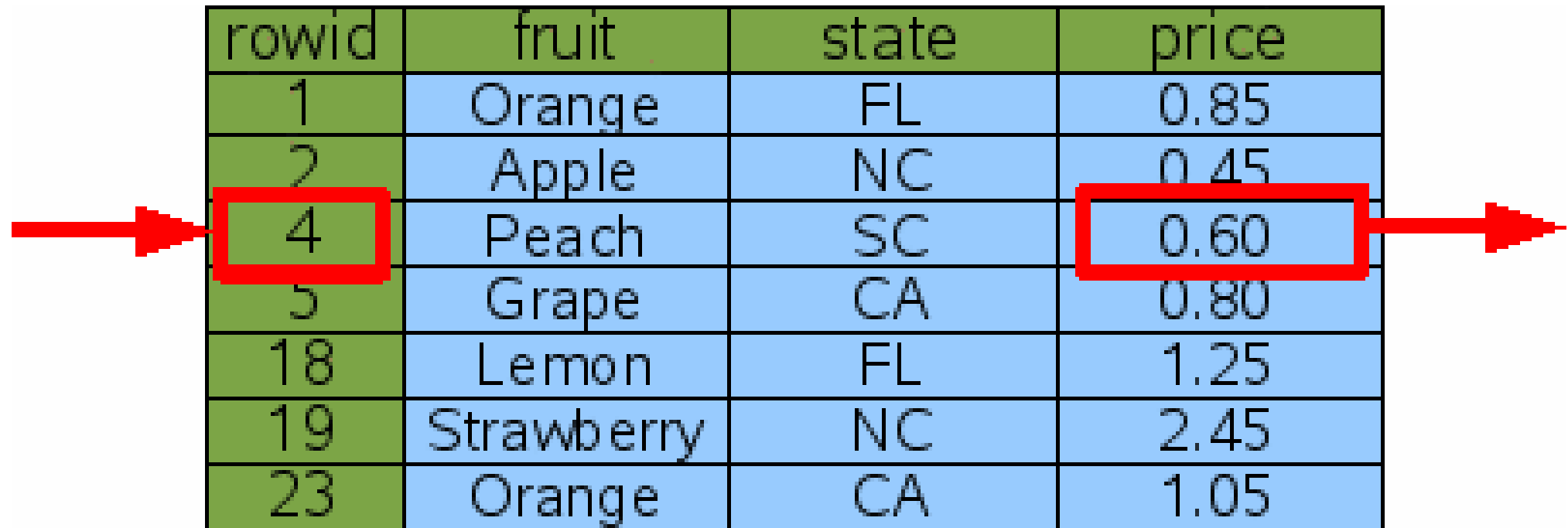
rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

Time grows with **N**.

Lookup by Rowid

```
select price from fruitsforsale  
where rowid = 4;
```

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05



Time grows with **log N**.

An Index

```
create index Idx1 on fruitsforsale(fruit);
```

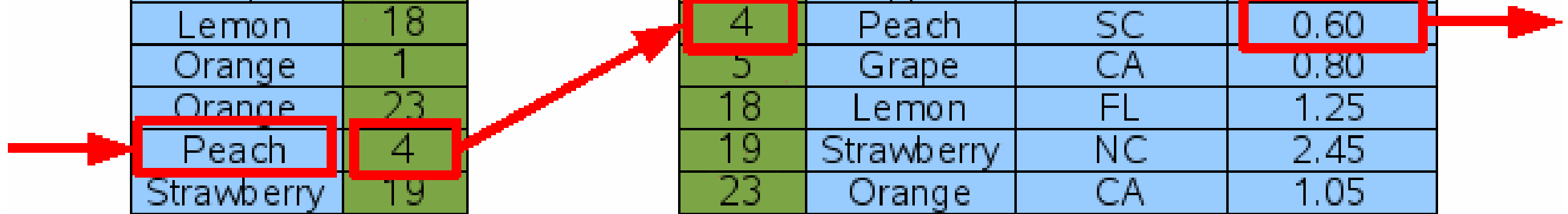
fruit	rowid
Apple	2
Grape	5
Lemon	18
Orange	1
Orange	23
Peach	4
Strawberry	19

A sorted copy of a column, plus rowids.

Using an Index

fruit	rowid
Apple	2
Grape	5
Lemon	18
Orange	1
Orange	23
Peach	4
Strawberry	19

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05



Two binary searches: the index, then the table.

Reading a Query Plan

```
explain query plan
select price from fruitsforsale
where fruit = 'Peach'
```

Plan text	Meaning
SCAN fruitsforsale	Read every row
SEARCH ... USING INDEX	Find rows by index
... COVERING INDEX	Index alone answers
USE TEMP B-TREE FOR ORDER BY	Sort step

Look for **SCAN** first.

Two Conditions

```
select price from fruitsforsale
where fruit = 'Orange' and state = 'CA';
```

fruit	rowid
Apple	2
Grape	5
Lemon	18
Orange	1
Orange	23
Peach	4
Strawberry	19

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

Find the oranges. Check their states.

A Multi-Column Index

```
create index Idx3 on fruitsforsale(fruit, state);
```

fruit	state	rowid
Apple	NC	2
Grape	CA	5
Lemon	FL	18
Orange	CA	23
Orange	FL	1
Peach	SC	4
Strawberry	NC	19

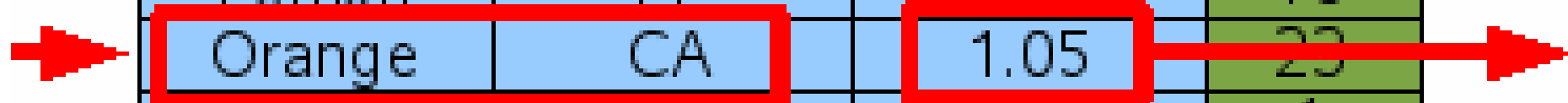
rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

Fruit first; state breaks ties.

A Covering Index

```
create index Idx4 on fruitsforsale(fruit, state, price);
```

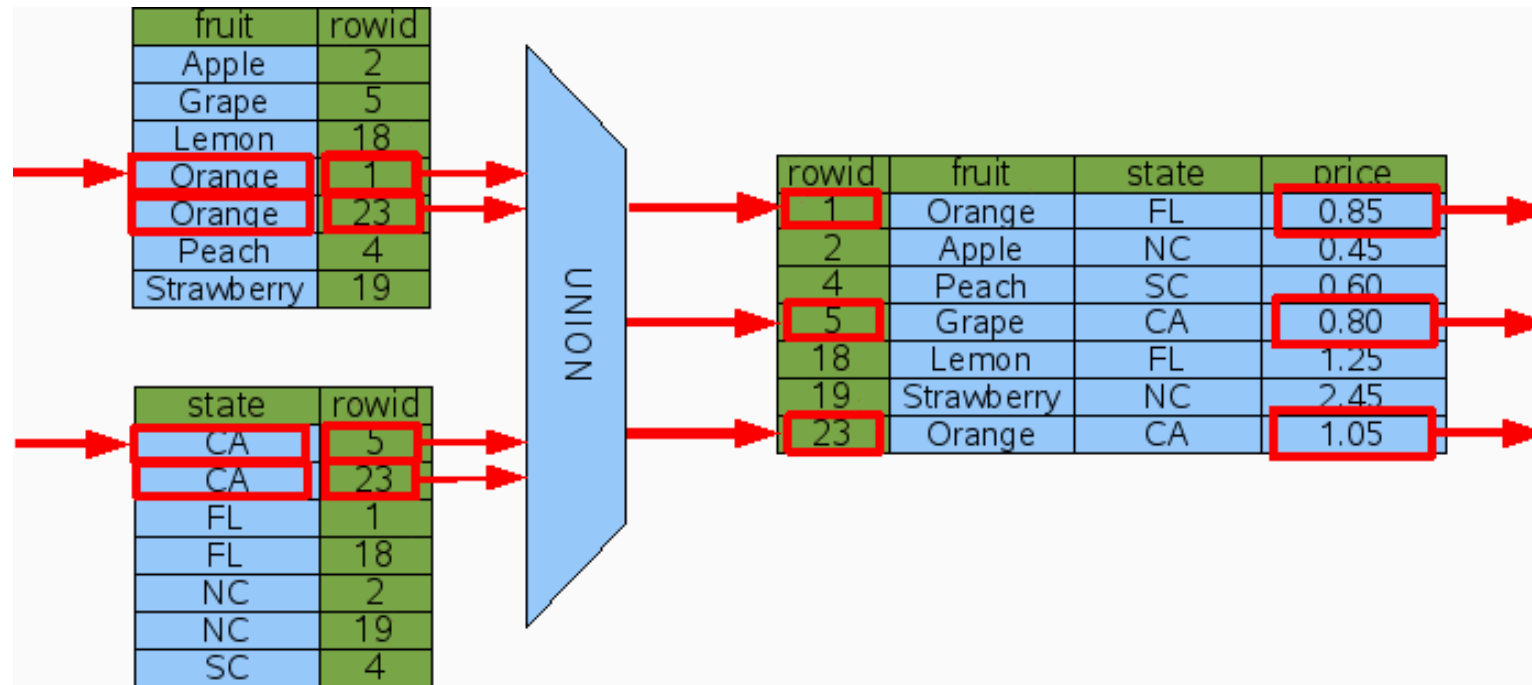
fruit	state	price	rowid
Apple	NC	0.45	2
Grape	CA	0.80	5
Lemon	FL	1.25	18
Orange	CA	1.05	23
Orange	FL	0.85	1
Peach	SC	0.60	4
Strawberry	NC	2.45	19



The index contains the answer: **price**.

OR

```
select price from fruitsforsale
where fruit = 'Orange' or state = 'CA';
```



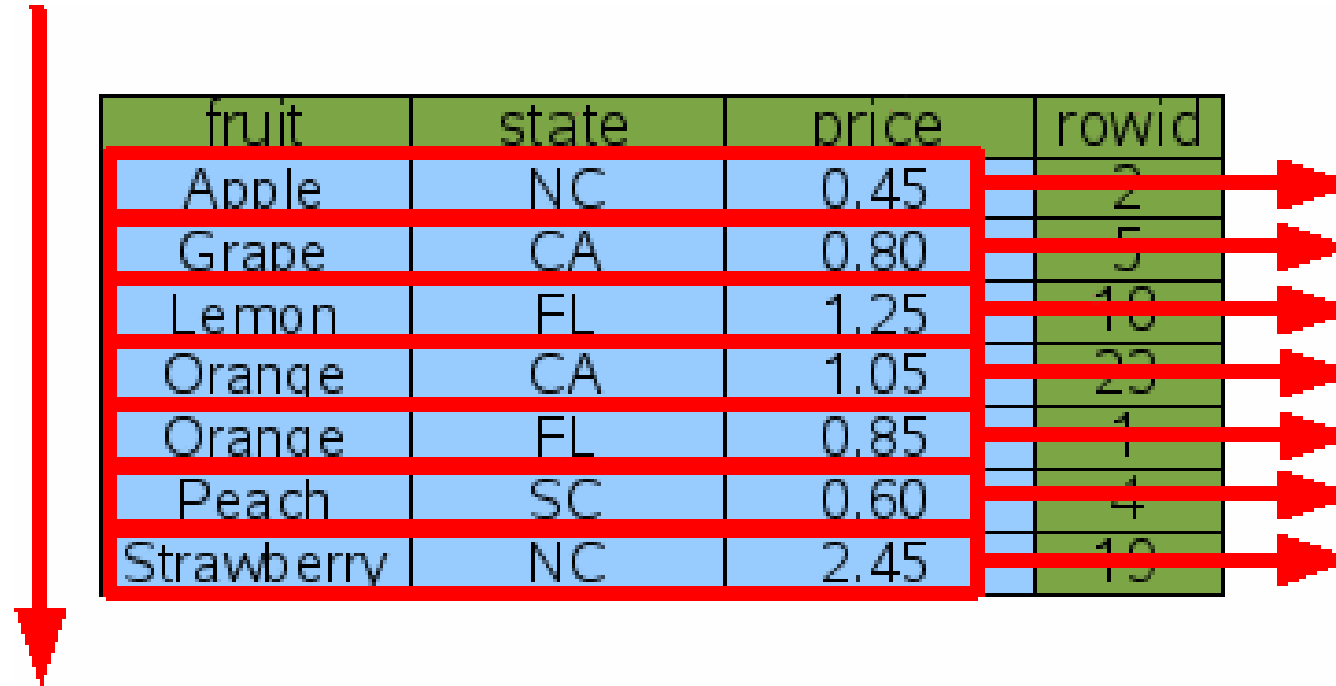
Combine matches; return each matching row once.

Sorting



Collect every row, then sort: **$K \log K$** , plus temporary storage.

Sorting with an Index



A diagram illustrating sorting with an index. A table with four columns (fruit, state, price, rowid) is shown. The first three columns are highlighted with a red box, and red arrows point from each of these columns to the rowid column, indicating that the data is being sorted based on the index values in the rowid column. A large red arrow points down from the title 'Sorting with an Index' to the table.

fruit	state	price	rowid
Apple	NC	0.45	2
Grape	CA	0.80	5
Lemon	FL	1.25	10
Orange	CA	1.05	23
Orange	FL	0.85	1
Peach	SC	0.60	4
Strawberry	NC	2.45	19

An index is already in order.

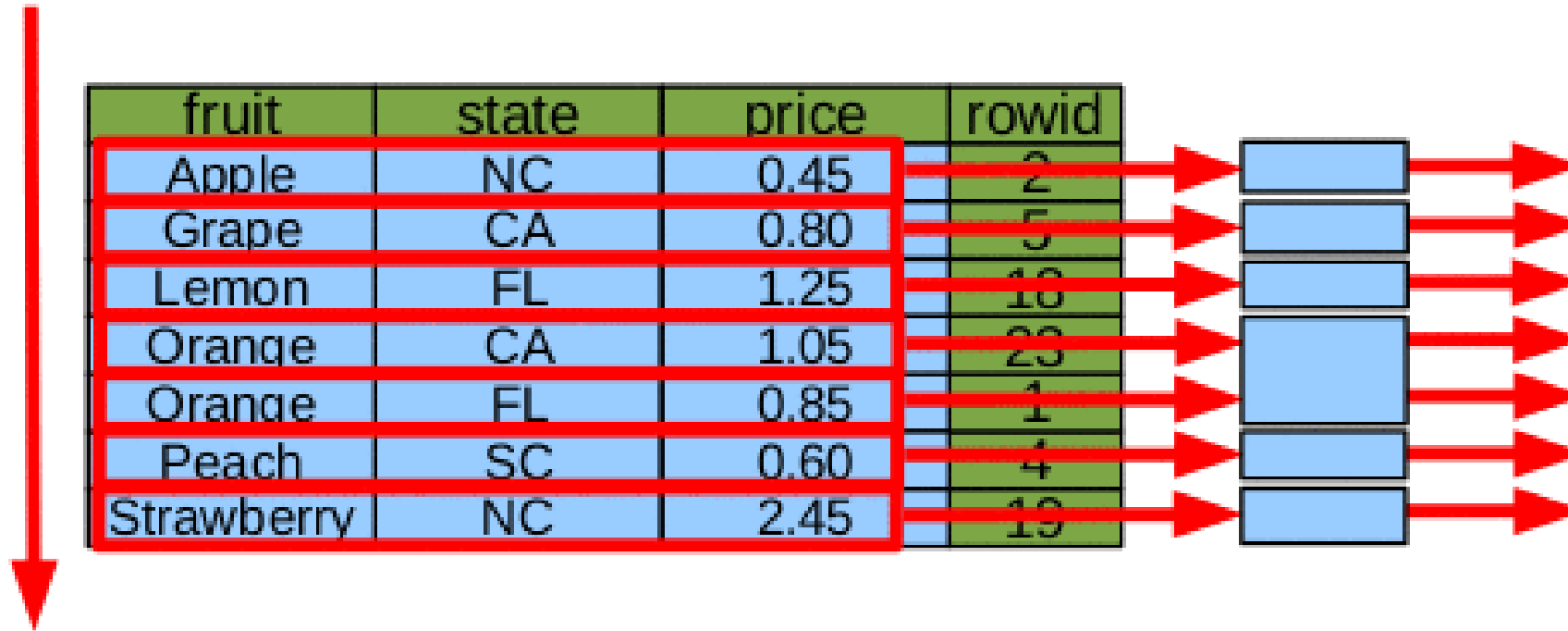
Search and Sort Together

fruit	state	rowid
Apple	NC	2
Grape	CA	5
Lemon	FL	18
Orange	CA	23
Orange	FL	1
Peach	SC	4
Strawberry	NC	19

rowid	fruit	state	price
1	Orange	FL	0.85
2	Apple	NC	0.45
4	Peach	SC	0.60
5	Grape	CA	0.80
18	Lemon	FL	1.25
19	Strawberry	NC	2.45
23	Orange	CA	1.05

where `fruit = 'Orange'` order by `state`, with an index on (`fruit`, `state`).

Partial Sort



order by fruit, price: the index gives fruit order; SQLite sorts within each fruit.

Statistics

```
analyze;
```

SQLite picks among indexes by estimated selectivity.

Without statistics, the choice is close to arbitrary.

It also makes a **skip-scan** possible.

Real Data

File	Rows
title.basics	12,811,750
title.ratings	1,713,431

Gzip-compressed tab-separated values (TSV). Missing value: \N.

Personal and non-commercial use only.

Keep the Data Out of Git

Before downloading, add to the root `.gitignore`:

```
*/data/  
*/imdb.db  
*/imdb.db-*
```

Commit the code and `.gitignore`. Keep the data local.

Check: `git check-ignore -v data/title.basics.tsv.gz`

Get Your Own Titles Database

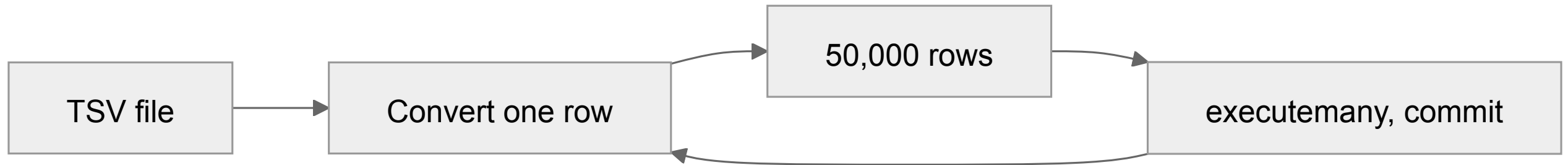
IMDb supplies **two compressed data files**.

The course supplies `import_imdb.py` and `schema.sql`.

1. Download `title.basics.tsv.gz` and `title.ratings.tsv.gz` into `data/`.
2. Run `python3 import_imdb.py` in the code folder.
3. Open the resulting `imdb.db`.

The loader reads local files; it does not download them.

Loading in Chunks



12.8 million rows in 54 seconds. No secondary indexes yet.

One Index, One Query

```
select tconst, start_year from titles
where primary_title = 'The Godfather'
```

	Plan	Time
No index	SCAN titles	4,536 ms
create index ... (primary_title)	SEARCH ... USING INDEX	under 0.1 ms

47 titles are called “The Godfather”.

What an Index Costs

Index	Build	Size added
primary_title	18.4 s	381 MB
title_type, start_year	16.4 s	263 MB
title_type, start_year, primary_title	30.5 s	536 MB
ratings(num_votes)	0.7 s	18 MB

500,000 inserts: 0.25 s with no index, 0.37 s with one, 1.06 s with three.

Two Conditions on Titles

```
select count(*) from titles
where title_type = 'movie' and start_year = 1994;
```

Index	Time
No useful index	4,435 ms
start_year	37.7 ms
(title_type, start_year)	0.1 ms

Column Order Matters

Conditions	Index used
title_type	First column
title_type and start_year	Both columns
start_year alone	None , before analyze

start_year alone: full walk of the index, 432 ms.

After analyze: skip-scan, 1.9 ms.

title_type alone: search, 18 ms.

Covering a Titles Query

```
select primary_title ... where title_type = ? and start_year = ?
```

	Time
Two-column index	5.0 ms
Covering index	1.4 ms

OR on Titles

```
select tconst from titles
where start_year = ? or primary_title = ?
```

	Time
One term unindexed	4,568 ms
Both indexed	0.1 ms

LIKE

Query	Index	Time
like 'The Godfather%'	Ordinary	4,669 ms, scan
like 'The Godfather%'	collate nocase	0.2 ms
like '%Godfather%'	Ordinary or nocase	5,024 ms, scan

A wildcard at the front cannot use an ordinary index.

Top Ten by Votes

```
select tconst, num_votes from ratings
order by num_votes desc limit 10
```

	Plan	Time
No index	SCAN + TEMP B-TREE	77 ms
Index on num_votes	SCAN ... USING INDEX	under 0.1 ms

The plan still says SCAN. The sort is gone, and limit stops the walk.

Python or SQL?

	Time
2,000 queries, one per title	148 ms
One join	10 ms
Fetch 643,991 years, count in Python	263 ms
group by in SQL, 15 rows	164 ms

Measure.

Stored Procedures

```
create or replace procedure  
    change_owner(pet integer, new_owner text)  
language plpgsql  
as $$ ... $$;  
  
call change_owner(1, 'Sam');
```

One call: read, update, log.

SQLite has none. This is **PostgreSQL**.

Choosing Indexes

Start from the queries.

Index what you search, join, and sort by.

Equality columns first, range columns last.

Drop an index that starts another.

Read the plan, then **measure**.

Exercise: Tune Three Queries

Longest movies over 300 minutes.

Titles rated 9.5 or higher with 10,000 or more votes.

Average rating of movies by decade.

Read the plan. Predict the index. Check.